

NAINA OS -- VOLUME 3

AI INTEGRATION FRAMEWORK & ARAL SPECIFICATION

ARAL (Doc 03.1), OpenClaw (Doc 03.2), Hermes (Doc 03.3) & Compatibility Matrix

DOCUMENT METADATA

Document Identifier:	NOS-AIIF-03.1 / 03.2 / 03.3
Specification Version:	1.0.0 (Production Engineering Spec)
Document Status:	APPROVED -- Open Standard
Date of Issue:	July 31, 2026
Author Group:	Chief Systems Architect & AI Runtime Group
Key Rule:	Architectural Rule #2 -- The Adapter Rule
Core Focus:	ARAL Contract * OpenClaw * Hermes * 13 Tech Compatibility Matrix

This document defines the formal AI Runtime Abstraction Layer (ARAL), OpenClaw Computer Use Engine, Hermes Conversational Engine, and the 13-Technology Compatibility Matrix.

NAINA OS -- AI Integration Framework (Volume 3: AIIF v1.0)

Document Revision History

Date	Revision	Author	Description of Changes
---	---	---	---
2026-07-31	v1.0.0	Chief Systems Architect	Release of Volume 3 -- AI Runtime A
2026-07-28	v0.9.0	AI Systems Group	Draft of Architectural Rule #2 & Sy

ARCHITECTURAL RULE #2: THE ADAPTER RULE

"No external technology, model, runtime, or framework shall interface directly with the NAINA OS Kernel. Every integration must pass through a strict, typed Stable Interface Adapter."

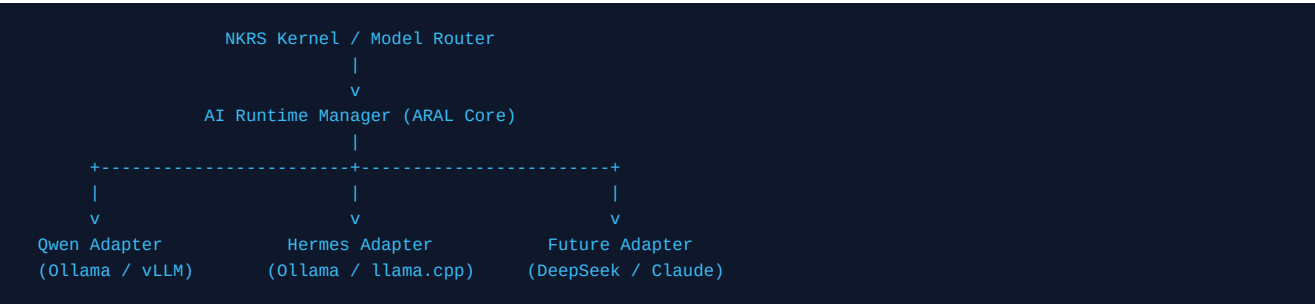
```
+-----+
|               THE ADAPTER PATTERN               |
+-----+
| CORRECT MANDATED DESIGN:                          |
| [NKRS Kernel] --> [Stable Interface (ARAL)] --> [Adapter] --> [External Project] |
+-----+
| PROHIBITED DIRECT COUPLING:                        |
| [NKRS Kernel] --x [External Project] (STRICTLY FORBIDDEN) |
+-----+
```

Rationale: External libraries, APIs, and models change APIs unpredictably or become obsolete. By enforcing Architectural Rule #2, replacing Qwen, Ollama, OpenClaw, or Hermes requires updating only its specific adapter module--zero lines of code in the NKRS Kernel are modified.

DOCUMENT 03.1 -- AI Runtime Abstraction Layer (ARAL v1.0)

1. Purpose & Core Design

The AI Runtime Abstraction Layer (ARAL) defines a uniform contract between the NAINA OS Kernel and all underlying LLM engines, Vision models, Embedding generators, and Speech runtimes.



2. Standard Capability Interface Specification

```

// ARAL Unified Model Adapter Interface (TypeScript / Python Contract)
export type CapabilityType =
  | 'CAP_CHAT'
  | 'CAP_STREAMING'
  | 'CAP_EMBEDDINGS'
  | 'CAP_VISION'
  | 'CAP_FUNCTION_CALLING'
  | 'CAP_JSON_MODE'
  | 'CAP_CONTEXT_WINDOW'
  | 'CAP_TOKEN_COUNTING';

export interface ChatMessage {
  role: 'system' | 'user' | 'assistant' | 'tool';
  content: string;
  images?: string[]; // Base64 or local URI for vision models
}

export interface ARALCompletionRequest {
  requestId: string;
  modelName: string;
  messages: ChatMessage[];
  temperature?: number;
  maxTokens?: number;
  responseFormat?: 'text' | 'json_object';
  tools?: Array<Record<string, unknown>>;
}

export interface IAIRuntimeAdapter {
  readonly adapterName: string;
  readonly supportedCapabilities: Set<CapabilityType>;

  // Core Capabilities
  chat(request: ARALCompletionRequest): Promise<string>;
  chatStream(request: ARALCompletionRequest, onChunk: (token: string) => void): Promise<void>;
  generateEmbeddings(text: string): Promise<number[]>;
  countTokens(text: string): Promise<number>;
  getMaxContextWindow(): number;
}
  
```

DOCUMENT 03.2 -- OpenClaw Computer Use Integration

1. Overview & Purpose

OpenClaw is integrated into NAINA OS as an optional, high-precision Computer Use Engine. Operating strictly under the command of CENANI, OpenClaw provides vision-guided desktop and browser automation without bypassing NAINA OS safety guardrails.

2. Computer Use Execution Flow

```
sequenceDiagram
    autonumber
    actor User
    participant NAINA as [NAINA] NAINA (Companion Engine)
    participant Planner as [AI] Goal Planner
    participant CENANI as [CENANI] CENANI (System Operator)
    participant Security as [SEC] Security Sandbox
    participant Adapter as [HOST] OpenClaw Adapter
    participant OS as [OS] Host OS / Browser

    User->>NAINA: "NAINA, open Chrome and download the sales report."
    NAINA->>Planner: Formulate multi-step task goal
    Planner->>CENANI: Hand off Task Execution Contract
    CENANI->>Adapter: Request GUI Action (Intent: OPEN_BROWSER)

    Adapter->>Security: Check Capability Token & Action Risk Level
    alt High Risk Action (e.g., Delete/Payment)
        Security-->>User: Trigger `ask_permission` UI Modal
        User-->>Security: User Approves Action
    end

    Security->>Adapter: Clearance Granted
    Adapter->>OS: Capture Screen -> Detect Coordinates -> Mouse/Keyboard Injection
    OS-->>Adapter: Screen State Updated
    Adapter-->>CENANI: Return Action Verification Status
    CENANI-->>NAINA: Task Complete Notification
```

3. Safety Confirmations & UI Change Recovery

- * UI Element Shift Handling: If a targeted button shifts position due to responsive layout change, OpenClaw re-samples the screen frame via Qwen-VL, re-computes spatial bounding boxes [ymin, xmin, ymax, xmax], and retries coordinate placement up to 3 times before raising an exception.
- * Safety Gate: Any action involving system settings modification, credential entry, or destructive file operations automatically calls the ask_permission kernel prompt.

DOCUMENT 03.3 -- Conversational Intelligence Layer (Hermes)

1. Overview & Architecture

Hermes (Nous Research Hermes 3) acts as the specialized Conversational Backend Engine powering NAINA's Personality Layer. While CENANI relies on mathematical code models, NAINA uses Hermes for long-context roleplay, high-EQ dialogue, and adaptive persona tracking.

```
User Voice Input
|
v
NAINA Personality Engine (System Persona Prompt & Working Memory)
|
v
AI Router (Detects CONVERSATIONAL / EMOTIONAL Intent)
|
v
Hermes Adapter (Ollama / llama.cpp GGUF Engine)
|
v
Streaming Natural Language Tokens -> Kokoro TTS Synthesizer
```

2. Features & Fallback Strategy

- * Long-Form Context Retention: Manages 32,000+ token context buffers without persona decay or instruction loss.
- * Replacement Strategy: If Hermes is replaced by a newer conversational model (e.g., Llama 4 Persona or Claude 3.5), only the HermesAdapter module is swapped out; the NAINA Personality Engine and NKRS Kernel remain untouched.

SYSTEM COMPATIBILITY MATRIX

Every integrated technology in NAINA OS is documented under this formal specification matrix:

Integrated Technology	Version Supported	Adapter Status	Required Capabilities	Known Limitations	Replacement Option	Maintenance Owner
Qwen (2.5 & VL)	v2.5 (7B/14B)	Core Engine	Reasoning, Code, Vis	Requires 8GB+ GPU	DeepSeek-Coder, Lla	AI Systems Team
Ollama	v0.5.0+	Core Runtime	GGUF Runner, REST	VRAM allocation lock	vLLM, llama.cpp nativ	Model Infra Team
OpenClaw	v1.2.0	Official Adapter	Screen Capture, Mou	High screen DPI scal	Playwright Native, Py	Automation Group
Hermes	Hermes 3 (8B/70B)	Official Adapter	High EQ, Persona, 32	Higher latency than 3l	Qwen-Instruct, Llama	HCI Persona Team
Faster-Whisper	v1.0.0 (Large-v3)	Core Engine	STT, VAD, CTranslate	Requires CUDA / Met	Whisper.cpp, Groq AF	Voice Subsystem Team
Kokoro / Piper	Kokoro-82M v1.0	Core Engine	Streaming TTS, 24kH	Single-threaded synth	Piper TTS, ElevenLab	Voice Subsystem Team
MCP	v1.0 (Anthropic)	Core Host	Tool Discovery, Execu	IPC serialization over	Custom gRPC Socket	Subsystems Group
Playwright	v1.40.0+	Core Service	Headless Chrome, DC	Heavy RAM footprint	Puppeteer, Selenium	Automation Group
ADB Bridge	v34.0.0 (Android)	Core Service	Touch, App Launch, L	USB/Wifi latency (~50	Appium Native Bridge	Mobile Systems Team
Obsidian	v1.5.0+	Core Service	Markdown Sync, Fron	Local disk I/O lock co	Logseq, Plain File Va	Memory Engine Group
LangGraph	v0.2.0+	Optional Adapter	Graph State Executio	High Python abstracti	Native NKRS Schedu	Agent Framework Group
CrewAI	v0.80.0+	Optional Adapter	Team Role Delegatio	Rigid agent communic	Native CENANI Mesh	Agent Framework Group
AutoGen	v0.4.0+	Optional Adapter	Multi-Agent Conversa	High token usage ove	Native Multi-Agent Rc	Agent Framework Group

End of NAINA OS AI Integration Framework -- Documents 03.1, 03.2, 03.3 & System Compatibility Matrix